

STUDENT PERFORMANCE ANALYZER

Python-Based Academic Performance Analysis and Visualization System

Project Report

Submitted as part of the flipped course evaluation

Student Name: S B Sudeepta

Roll Number: 26BAI10408

Course/Subject: Python Essentials

Academic Year: 2026–27

1. Introduction

Student Performance Analyzer is a Python application designed to convert raw student marks into useful academic information. The system reads records from a CSV file, validates the values, calculates averages and performance statistics, identifies top students, and creates charts using Matplotlib. The project demonstrates practical use of Python programming, file handling, functions, modular design, data processing and visualization.

2. Problem Statement

Manual analysis of student marks requires repeated calculations and makes comparison between students and subjects inconvenient. A simple computerized system is needed to process marks consistently and present results in an understandable form.

3. Objectives

- Read student data from a structured CSV file.
- Validate marks before processing.
- Calculate individual and class-level performance.
- Identify top-performing students.
- Calculate subject-wise averages.
- Generate graphical reports.
- Demonstrate modular Python programming and testing.

4. Functional Requirements

The system contains three major functional modules:

- 1. Data Management:** reads CSV records and validates them.
- 2. Performance Analytics:** calculates averages, pass/fail counts, highest/lowest performers and top students.
- 3. Reporting & Visualization:** displays results and creates Matplotlib charts.

Input: CSV file containing RollNo, Name, Python, Maths and Physics marks.

Output: terminal reports and PNG charts.

5. Non-Functional Requirements

- **Performance:** processes small/medium CSV files quickly.
- **Usability:** menu-driven interface with simple choices.
- **Reliability:** handles missing files and invalid marks safely.
- **Maintainability:** functionality is separated into modules.
- **Resource Efficiency:** uses lightweight CSV storage instead of a database for this project scale.
- **Error Handling:** invalid records are rejected and file errors are reported.

6. System Architecture

The system follows a modular architecture. The main program manages user interaction. The data manager reads the CSV file and sends records through the validator. The analytics module performs calculations. The reports module prints summaries, while the visualizer creates charts. This separation

improves readability and maintainability.

7. Design Diagrams

The repository contains Use Case, Workflow, Component/Class and Sequence diagrams in the **docs/** folder. The storage design is a CSV schema because the project does not require a relational database.

8. Design Decisions & Rationale

CSV was selected because it is easy to create, inspect and process using Python. A command-line interface keeps the project simple and suitable for demonstrating core Python concepts. Matplotlib was selected for visualization because it integrates directly with Python and can create bar charts without a complex framework.

9. Implementation Details

The project is divided into meaningful modules: `main.py`, `data_manager.py`, `validators.py`, `analytics.py`, `reports.py`, `visualizer.py` and `config.py`. The tests folder contains automated tests. The application uses functions, dictionaries, lists, file handling, exception handling and sorting. Matplotlib is used to save charts in the output folder.

10. Screenshots / Results

When executed, the application displays a menu with options for summary, top students, subject averages and chart generation. Chart images are saved under `output/`. The supplied sample dataset contains 10 students and three subjects.

11. Testing Approach

Unit tests are provided using Pytest. Tests cover record validation, class statistics, top-student sorting and subject-average calculation. Invalid marks outside the 0–100 range are rejected by the validator.

12. Challenges Faced

Key challenges include handling CSV values as strings, validating incomplete records, calculating averages correctly, separating responsibilities between modules and saving charts with readable labels. These were addressed using validation functions, exception handling and modular design.

13. Learnings & Key Takeaways

The project provides practical understanding of Python functions, modules, CSV file handling, data validation, sorting, calculations, exception handling, visualization and basic software project organization. It also demonstrates how a real-world problem can be divided into smaller technical modules.

14. Future Enhancements

Possible future versions can add a graphical user interface, database storage, student login, exportable PDF reports, more subjects, attendance analysis, trend analysis and machine-learning-based performance prediction.

15. References

Python documentation; Matplotlib documentation; Pytest documentation; course notes and classroom material on Python, CSV, data handling and visualization.

Appendix A – Sample Input

RollNo	Name	Python	Maths	Physics
RollNo	Name	Python	Maths	Physics
101	Aarav Sharma	92	88	90
102	Diya Patel	85	91	87
103	Rohan Verma	76	79	72
104	Ananya Singh	95	93	96
105	Arjun Mehta	68	74	70
106	Meera Joshi	88	84	90
107	Kabir Khan	72	69	75
108	Ishita Gupta	91	89	94
109	Aditya Rao	63	71	66